Recommending films by low-rank matrix completion: alternating least squares versus singular value thresholding

Interim report 1: scoping memo

Augusta Ada Lovelace
Charles Babbage
Linear Algebra, UCU Data Science Master Programme, October 18, 2026

Domain	Recommender systems
Method family	SVD and low-rank approximation
LA object	The 943×1682 ratings matrix, about 6% of entries observed, approximated by a matrix of rank $k \leq 20$
Algorithms	Alternating least squares (own implementation); singular value thresholding (own iteration, library SVD)
Data	MovieLens 100K [3]

1 Problem and motivation

Guidance. *Criterion A. What goes in, what comes out, and why it matters in practice. Who uses a solution to this problem, and what goes wrong without a good one? Give a short overview of the approaches in the literature and say which ones you chose and why.* Interim report 1*: half a page.*

Your introduction goes here.

2 Linear-algebra formulation

Guidance. *Criterion A. State the problem precisely as a linear-algebra problem: the matrix (its size, its entries, its structure: sparse, symmetric, positive definite, low rank), the unknowns, and what you need to do with the matrix. Explain why this structure suggests your methods.* Interim report 1*: this section is the core of the memo.*

Number every displayed formula you refer to, and refer to it by number, as in (1) below. Use `\begin{equation}` or `\begin{align}`, never `$$ ... $$`. For example, the least squares problem with a matrix $A \in \mathbb{R}^{m \times n}$ of full column rank and its solution via the thin QR factorization $A = QR$ read

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2, \qquad R\mathbf{x} = Q^\top \mathbf{b}. \tag{1}$$

Definition 2.1. The condition number of a matrix A of full column rank is $\operatorname{cond}(A) = \sigma_{\max}(A)/\sigma_{\min}(A)$.

Remark 2.2. Since $\operatorname{cond}(A^\top A) = \operatorname{cond}(A)^2$, solving (1) through the normal equations can lose twice as many digits as solving it through QR.

3 Methods

Guidance. *Criteria B and C. At most two algorithms. For each: the key idea, explained in your own words; the central claims, proved or sketched (say which, and cite the source of any proof you do not give); the cost in flops and memory, as a function of the problem size; and the conditions under which it fails or becomes slow.* Interim report 1*: the candidate approaches, their pros and cons, and their predicted cost.*

Algorithm 1: Power iteration for the dominant eigenpair

Input: $A \in \mathbb{R}^{n \times n}$, starting vector $\mathbf{x}_0 \neq 0$, tolerance $\varepsilon > 0$
Output: approximate dominant eigenpair $(\lambda, \mathbf{x})$

$\mathbf{x} \leftarrow \mathbf{x}_0 / \|\mathbf{x}_0\|$; $\mathbf{y} \leftarrow A\mathbf{x}$; $\lambda \leftarrow \mathbf{x}^\top \mathbf{y}$;
while $\|\mathbf{y} - \lambda \mathbf{x}\| > \varepsilon\,|\lambda|$ **do**
  $\mathbf{x} \leftarrow \mathbf{y} / \|\mathbf{y}\|$;
  $\mathbf{y} \leftarrow A\mathbf{x}$; // one matrix-vector product: $\sim 2\,\mathrm{nnz}(A)$ flops
  $\lambda \leftarrow \mathbf{x}^\top \mathbf{y}$; // Rayleigh quotient: $2n$ flops
return λ, $\mathbf{x}$

3.1 Algorithm 1

Present an algorithm either as pseudocode, as in Algorithm 1, or as a short code listing, as in Listing 1. Annotate the expensive steps with their cost.

Cost. About $2\,\mathrm{nnz}(A)$ flops per iteration and $O(n)$ memory besides A. The error decreases like $|\lambda_2/\lambda_1|^k$, so the iteration is slow when the two largest eigenvalues are close in absolute value.

Listing 1: Power iteration in NumPy.

```python
import numpy as np

def power_iteration(A, x0, tol=1e-10, maxit=1000):
    x = x0 / np.linalg.norm(x0)
    y = A @ x                      # the only expensive step
    lam = x @ y
    for _ in range(maxit):
        if np.linalg.norm(y - lam * x) <= tol * abs(lam):
            break
        x = y / np.linalg.norm(y)
        y = A @ x
        lam = x @ y
    return lam, x
```

3.2 Algorithm 2

The second algorithm, if you compare two.

3.3 Cost and failure modes

Guidance. *Compare the two algorithms side by side: cost per step, number of steps, memory, sensitivity to conditioning, and the regime (size, sparsity, rank) in which each one wins. Section 4 then checks these predictions.*

4 Implementation and experiments

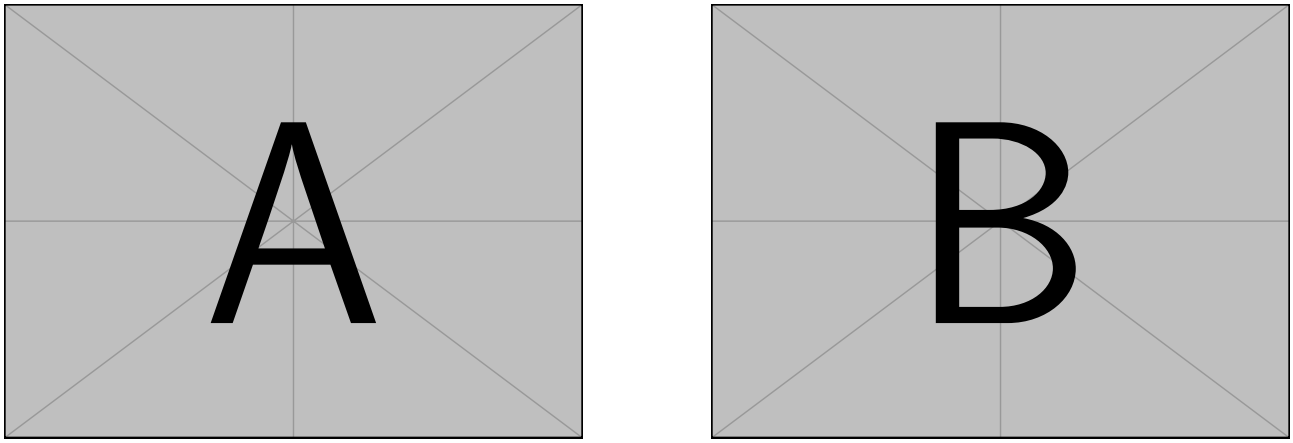
Guidance. *Criterion D. What you implemented yourselves, and what you called from a library. The data, the benchmark and the metrics. Then the results: accuracy, the condition numbers of the matrices you actually factor, runtime against problem size next to the predicted cost, and the size or condition number at which a method stops working.* Interim report 2*: a first implementation running on small data and one comparison with a library routine, with one figure.*

4.1 Setup

Describe the hardware and the software versions briefly, so that a reader can judge the timings. Every figure and table must be reproducible from the `README` of your repository.

4.2 Results

Figure 1 shows how to include a figure, and Table 1 how to report a comparison. Save plots from Python as vector PDF, with `plt.savefig("fig.pdf")` rather than as PNG. Label both axes, with units, and use a logarithmic axis when values span several orders of magnitude.



(a) Runtime against n.

(b) Relative error against cond(A).

Figure 1: A figure with two panels. The caption says what the reader should see, not only what is plotted.

Table 1: Comparison of methods. Report the mean and the spread over several runs, and say how many runs.

Method	n	Time (s)	Relative error
Algorithm 1 (ours)	...	...	...
Algorithm 2 (ours)	...	...	...
Library routine	...	...	...

5 Discussion and limitations

Guidance. *Criteria C and D. Did the experiments confirm the predicted cost and failure modes? Explain every surprise. What did not work, and why? An honest explanation of a failure earns credit; a clean result with no explanation does not.*

6 Conclusion

Guidance. *What you achieved, the main difficulties, and what you would do next. Half a page is enough.*

7 Plan and division of work

Guidance. *Interim reports only; this section disappears from the final report.* Interim report 1*: the planned steps and who does what.* Interim report 2*: what is not working yet, any change of plan, and the outline of the final report.*

References

[1] G. Strang, *Linear Algebra and Learning from Data*, Wellesley–Cambridge Press, Wellesley, MA, 2019.

[2] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, Baltimore, MD, 2013.

[3] F. M. Harper and J. A. Konstan, The MovieLens datasets: history and context, *ACM Transactions on Interactive Intelligent Systems* **5**(4), article 19, 2015. doi:10.1145/2827872.

[4] C. R. Harris et al., Array programming with NumPy, *Nature* **585**, 357–362, 2020. doi:10.1038/s41586-020-2649-2.